Log Compression and Rule Learning for Root Cause Analysis of RISC -V Build Failures

Zehua Wang
Institute of Software, Chinese Academy of Sciences
University of Chinese Academy of Sciences
China

Abstract

In recent years, RISC -V has been rapidly developing, leveraging its openness, modularity, and extensibility, and is becoming an important architectural choice for both general-purpose and domain-specific processors. A large number of open-source software packages are being ported to the RISC -V platform, while at the same time revealing frequent and complex build failures that repeatedly occur as upstream evolves. Build logs are often extremely large, structurally heterogeneous, and noisy; the truly decisive clues for failures are usually buried in output segments from different stages that are far apart from each other. Traditional manual investigation or rule-based matching is inefficient and costly, and struggles to support large-scale migration practices.

This paper targets build failures of RISC -V software packages and proposes a "log compression + rule learning"-based method for root cause analysis. First, we design a Multi-Granularity Semantic Compression method (MGSC) oriented to long-range causal dependencies in logs. MGSC performs explicit error anchor localization, entity-based long-distance context backtracking, and lightweight semantic compression under a strict *token* budget, so as to preserve high-fidelity key information that is relevant to the root cause. Then, we propose Historical Rule Learning for Root Cause Analysis (HRL-RCA), which learns structured repair rules from historical fixes, builds a rule-tree index organized by error type and build stage, and combines it with the compressed log summaries to perform conditional matching and reasoning, ultimately outputting interpretable root cause classifications and repair suggestions.

Experiments conducted on a RISC -V build-failure dataset collected from a real OBS production environment show that the proposed method outperforms multiple baselines in key error-line coverage, root cause identification accuracy, and quality of repair suggestions, while maintaining a good balance between log compression ratio and inference overhead.

Keywords

RISC -V, build failure, root cause analysis, log compression, large language model

1 Introduction

In recent years, RISC -V has been rapidly developing thanks to its openness, modularity, and extensibility [3], and is gradually becoming one of the major options for next-generation general-purpose and domain-specific computing. With continuous investments from both industry and academia, a large number of mainstream open-source software projects are being ported to the RISC -V architecture [12]. However, since the upstream of mainstream open-source software has long been centered on x86 and ARM, their continuous version iterations keep introducing new architecture-specific features. As a result, many packages that were once adapted to RISC -V fail to build again after following upstream updates [10].

In distribution-level package migration practices, platforms such as the Open Build Service (OBS) provide unified infrastructure for cross-distribution and cross-architecture package builds through automated building, dependency management, and release processes. However, in such highly automated pipelines, build failures exhibit even higher frequency and diversity: the same package may fail on different architectures, with different dependency versions, or at different build stages (e.g., `%prep`, `%build`, `%install`, `%check`). These failures are often caused by a combination of factors such as architectural differences, dependency-chain constraints, build script configurations, and patch changes. Build logs are typically huge (tens of thousands to hundreds of thousands of lines), while truly failure-related key information accounts for less than 5%, and log formats vary greatly across packages and stages. More importantly, the final error message is often only the consequence, while the real root cause might have appeared much earlier in the pipeline, forming long-range causal dependencies.

To more intuitively illustrate such asynchronous triggering and long-range causal dependency patterns, Figure 1 shows a real build-log snippet for the `aide` package. The failure presents a typical long-distance distribution in the log: (1) *Build termination signal (tail)*: the build system eventually throws a generic termination signal (`RPM build errors`) at line 1021. (2) *Explicit error statement (middle)*: looking backwards, the real compilation error appears at line 998, where the log reports `` `GCRY_MD_SM3' undeclared ``. (3) *Implicit root cause (head)*: however, the root cause of this undefined macro hides in the dependency installation stage hundreds of lines earlier—at line 471 the system installs `libgcrypt-1.8.7-1.oe1`, a version that does not support the SM3 algorithm. Note that line 471 is just a normal `installing` line with no anomaly or error mark at all.

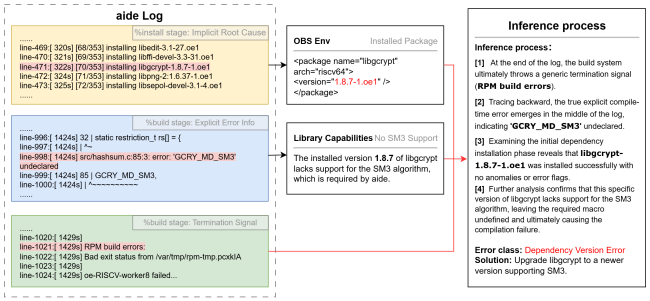

Figure 1: Example of long-range causal dependencies in build logs

This "cause first, error later" phenomenon reveals the core difficulty of handling ultra-long build logs: if automated analysis only relies on taking a local window around the final error, or simply filters lines by keywords such as `error`, the decisive dependency-version clue at line 471 will be completely discarded. This leads to **Challenge 1**: how to locate the true error from an ultra-long log—key error lines are buried in massive noise, while the real root cause may have appeared hundreds of lines earlier, and naive windowing around the final error easily misses causal clues.

Even if the log compression stage successfully retains the key evidence chain, root cause analysis and repair recommendation still suffer from lack of reliable historical experience. A naive vector-similarity-based RAG often encounters *similarity misleading*: similar logs do not necessarily imply applicable fixes. As shown in Figure 2, the build logs of the `apache-mime4j` package and the `hibernate` package share exactly the same key error message, but by further considering their respective build scripts and environmental constraints, we find that their effective fixes are actually different. This case shows that the fix for a build failure depends not only on the error text itself, but also heavily on contextual prerequisites such as build stage, toolchain, and dependency combination. Solely relying on textual similarity to retrieve historical cases tends to bring in examples that do not satisfy the applicability conditions, which increases the risk of misdiagnosis.

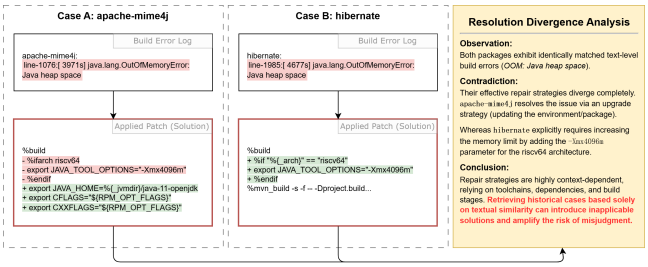

Figure 2: Misleading example: similar errors but different fixes

In addition, the main difficulty for large language models on such tasks is not lack of expressive capability, but insufficient verifiability of their reasoning results. When multiple pieces of evidence coexist, or when evidence fragments are missing or mutually inconsistent, models tend to generate fluent but hard-to-reproduce explanations and solutions. In engineering troubleshooting, repair suggestions must be reviewable and reproducible: they should not only state the cause, but also explicitly specify which log evidence they rely on, under what preconditions they hold, and which historical fix records they correspond to. This gives rise to **Challenge 2**: how to retrieve the *right* historical fixes—repair experience is scattered across commits and issue discussions in different repositories; similar logs do not imply applicable fixes, and differences in version, dependencies, and build stage may lead to misleading recommendations.

Therefore, for RISC -V package-migration build failures, we urgently need an automated approach that can, under limited token-context budgets, preserve high-fidelity evidence chains and organize historical repair experience into verifiable, condition-based, and evidence-traceable knowledge.

To address the above challenges, this paper makes the following contributions:

(1) We propose Multi-Granularity Semantic Compression (MGSC), a log compression method oriented to long-range causal dependencies. MGSC follows a three-stage pipeline of "explicit error anchor localization—entity-based long-distance context backtracking—lightweight semantic compression", and preserves high-fidelity evidence chains under strict token budgets.
(2) We propose Historical Rule Learning for Root Cause Analysis (HRL-RCA). HRL-RCA structures historical repair experience into rules and organizes them into a tree index along the axes of "error type—build stage—applicability conditions". During online inference, it performs conditional routing to retrieve rules that satisfy their preconditions and, under a ReAct-style framework, generates traceable root cause analyses and repair suggestions.
(3) Based on an OBS production environment, we construct a RISC -V build-failure dataset and conduct systematic experiments. Results show that MGSC achieves higher coverage of key evidence than existing compression methods, while HRL-RCA attains a comprehensive diagnostic score of 79.0%, significantly outperforming various baselines, and reduces processing time by about 61%.

2 Related Work

2.1 Log Parsing and Semantic Compression

Log parsing aims to normalize unstructured logs into structured representations of event templates and dynamic parameters. Early work relied on rules and regular expressions. Representative methods include SLCT [17], which mines log patterns via clustering, Drain [7], which builds a fixed-depth parsing tree for efficient online parsing, and Logram [4], which uses *n*-gram dictionaries to accelerate template recognition. With the rise of deep learning, LSTM-based methods such as DeepLog [5] learn temporal patterns of normal execution paths for anomaly detection, while Transformer-based and self-supervised approaches provide stronger semantic representations [2, 6]. However, these methods often struggle to generalize across systems and versions.

Recently, large language models (LLMs) have brought a new paradigm to log parsing [1]. The few-shot log-parsing method LogPrompt [9] reduces cost by careful example selection; DivLog [19] enhances cross-domain parsing via in-context learning; LLMParser [11] treats LLMs as generative parsers and provides empirical analysis. LogSage [20], a log analysis framework for CI/CD scenarios, proposes token-efficient preprocessing and structured diagnosis. However, in build-failure scenarios of software packages, where long-range causal dependencies and cross-stage reasoning are crucial, how to stably preserve traceable evidence chains under tight token budgets remains an open problem requiring task-specific compression methods.

2.2 Log-based Root Cause Analysis

Root cause analysis (RCA) aims to identify underlying causes of failures from system outputs. Traditional approaches often use rule-based correlation to match events [13], or combine with source-code analysis to infer failing execution paths [21]. Deep learning methods cast RCA as a log-line relevance ranking task, extracting indicative descriptions and parameters from logs for structured diagnosis [18].

LLM-based RCA emphasizes directly generating human-readable diagnostic conclusions from failure logs. Roy et al. [15] show that agent-based designs can alleviate the sensitivity of single-turn QA to context budgets and missing information. Shuai et al. [16] propose a two-stage framework in which RV-LAD performs log compression, and MCTS-RCA combines Monte Carlo tree search with RCA to guide LLM reasoning. However, approaches that rely solely on "similar-log retrieval + free-form generation" risk introducing misleading evidence and amplifying hallucinations; applicability conditions, evidence alignment, and verification mechanisms must be incorporated in a systematic way. Our method explicitly learns verifiable rules from historical repair experience and organizes them into a rule tree with conditional routing, thereby reducing the risk of misleading retrieval at its source.

2.3 Build Failure Analysis and Package Migration

Beyond general log analysis and RCA, there is also prior work specifically focusing on build failures. One line of research conducts empirical studies and pattern mining on build failures in large open-source ecosystems [14], aggregating failure records from distribution repositories and CI pipelines to characterize typical failure types, error-prone stages, and their correlations with dependency relationships and configuration scripts, thereby providing empirical foundations for automated repairs. Another line of work, from a more engineering-oriented view, explores using classification models or triage methods in CI build scenarios to automatically attribute and assign build failures [8], reducing manual debugging effort.

However, these build-failure studies mostly remain within a "short window + local features" paradigm: they either rely on manually designed error patterns and heuristics, which struggle with heterogeneous logs and cross-version evolution; or they directly truncate long logs into a single model input, ignoring cross-stage long-range causal dependencies. In addition, many existing systems focus on error-type prediction or simple heuristic recommendations, and lack structured organization of historical repair knowledge and explicit modeling of applicability conditions. In contrast, our work introduces both MGSC, which preserves evidence chains in long logs, and HRL-RCA, which models historical repair knowledge as a rule tree with explicit conditions, achieving in a unified framework efficient long-log processing, cross-stage root cause localization, and verifiable repair recommendations.

3 Method

We adopt a two-stage pipeline for handling RISC -V build failures. In the first stage, MGSC performs multi-granularity semantic compression on ultra-long logs and outputs a structured summary centered on key error lines with necessary context. In the second stage, HRL-RCA takes the compressed summary as input, combines it with a rule-tree index built from historical repair experience, and generates traceable root cause analyses and repair suggestions via conditional routing and ReAct-style reasoning.

3.1 MGSC: Multi-Granularity Semantic Compression

Log compression for build failures is not merely about dropping most lines; instead, the goal is to preserve, under a limited budget, the key evidence that supports root cause analysis. To this end, we propose Multi-Granularity Semantic Compression (MGSC), a method oriented toward long-range causal dependencies in logs. The core idea is to first quickly locate failure manifestations using explicit error information as anchors, then backtrack triggering conditions across long distances using key entities as clues, and finally perform fine-grained semantic compression and structuring over the candidate evidence set. In this way, the original long log is transformed into a summary that does not lose causal information. MGSC emphasizes two aspects: multi-granularity and long-range associations. Multi-granularity means progressively focusing from key error lines, to error-related segments, and further down to entity-level clues; long-range associations are achieved by using entity backtracking to reconnect distant causal clues that would otherwise be lost if we only kept the tail region around the final error. As shown in Figure 3, MGSC consists of three stages: Stage 1 uses lightweight rules and heuristic scoring to prioritize segments near errors and critical turning points in the build process, obtaining candidate error blocks with high recall; Stage 2 extracts key entities from candidate blocks and backtracks them across the entire log to complete the causal chain; Stage 3 performs fine-grained semantic compression and structuring over the candidate segments, yielding a high-density information carrier under a fixed budget.

Given an original build log $L = \{l_1, l_2, \ldots, l_N\}$, the goal of MGSC is to produce a structured compressed result S that satisfies a token-budget constraint $|S| \leq B$ while maximizing the coverage of key evidence and causal clues in S. Here B is typically set to half of the downstream LLM's context window, leaving sufficient space for subsequent root cause reasoning and tool calls.

3.1.1 Fast Coarse-Grained Filtering. RISC-V package build logs on OBS are typically ultra-long, heterogeneous, and noisy. On the one hand, logs contain massive normal compilation outputs, platform messages, and toolchain information, while the content truly related to the root cause only accounts for a tiny fraction, resulting in a very low signal-to-noise ratio. On the other hand, the build process spans multiple stages, and triggering conditions may first appear in early probing or dependency-checking stages, but manifest as explicit errors later during compilation or linking. Simply applying entity backtracking or semantic compression to the full log is not only computationally expensive and token-costly, but also prone to being disturbed by redundant information. Therefore,

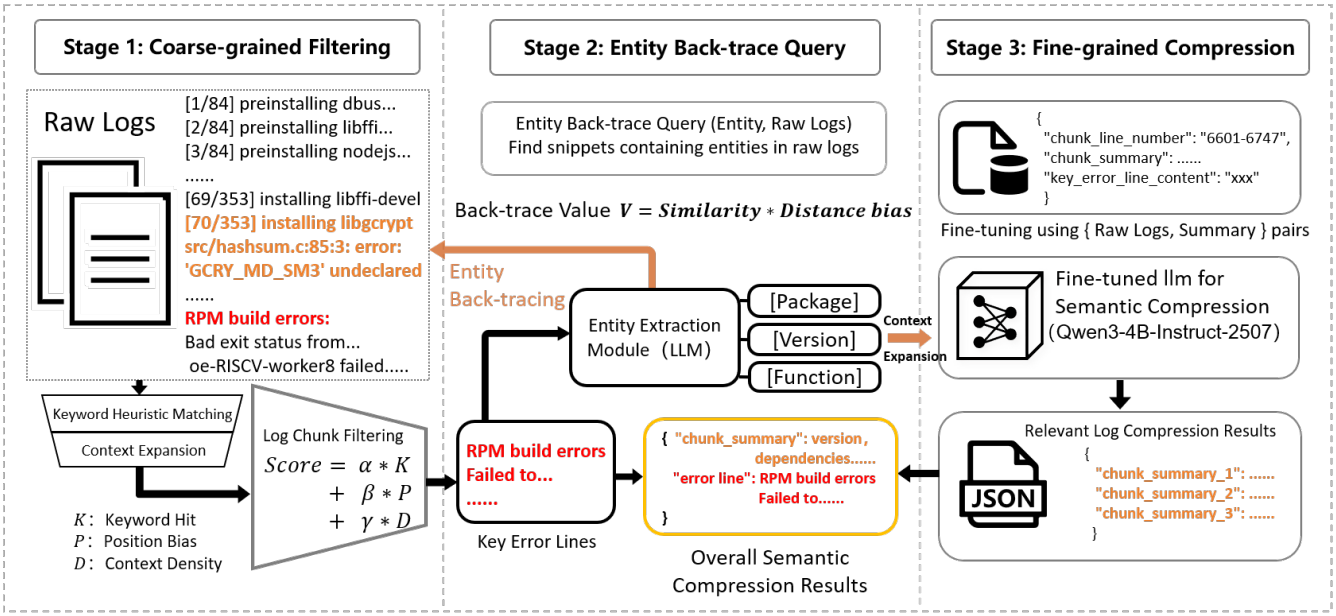

Figure 3: Overall workflow of MGSC: Multi-Granularity Semantic Compression

MGSC first performs a fast coarse-grained filtering stage, which quickly removes verified useless outputs using lightweight rules and heuristics, locates explicit error anchors, and generates a small set of highly relevant candidate blocks centered around those anchors.

Concretely, we first use a set of regular-expression rules R_{drop} to remove redundant outputs that repeatedly appear across packages but are unrelated to failures (e.g., compiler version echoes, download progress, fixed-format status messages), yielding a filtered log $L' = \{l_i \in L \mid g(l_i) = 1\}$. We then perform multi-level keyword matching to collect a set of key error lines A, with matching layers including: (1) strong error patterns such as `error:`, `fatal`, `undefined reference`, `No such file`, and `Bad exit status`; (2) build termination messages such as `RPM build errors` and `make: *** Error`; and (3) supplementary signals such as `undefined symbol` and `cannot find -lxxx`. Duplicate errors and repeated templates are merged to avoid a single error occupying too much budget.

After obtaining the key error lines A, we expand a window around each error line to form candidate blocks $\mathcal{B}$, ensuring local semantic completeness. For each line l_i in a block, we compute a line-level importance score:

$$S(l_i) = \alpha \cdot I_{key}(l_i) + \beta \cdot f_{pos}(i) + \gamma \cdot D_{context}(i) \tag{1}$$

where I_{key} is an indicator function for keyword matches, $f_{pos}(i) = e^{-\lambda(N-i)/N}$ is a positional bias term (lines closer to the log tail receive higher weights), and $D_{context}$ is a local keyword-density term. Block-level scores are obtained by aggregating line scores (e.g., max + mean), ensuring that blocks containing strong error anchors are ranked higher while avoiding an unfair advantage for longer blocks. The system ranks blocks by their scores and selects the Top-n blocks under the token budget. In our setting, the total token length of this stage is constrained to be no more than one quarter of the downstream LLM's context window, leaving ample room for subsequent entity backtracking and semantic compression.

3.1.2 Entity-Based Context Backtracking. After coarse-grained filtering, the system has obtained a set of highly relevant candidate blocks with explicit error lines serving as anchors and preserving the local context where failures occur. However, in RISC-V build-failure scenarios, relying solely on windowed snippets around errors is often insufficient for stable root cause analysis because of pronounced long-distance effects: the true triggering clues may appear earlier in `configure` checks, dependency installations, or configuration information, while final errors break out during `%build`; the same root cause may manifest multiple times in different forms, for example, an early warning about missing headers/libraries, later compilation errors about undefined types/symbols, and final link-time `undefined reference` messages. This "cause first, symptoms later" long-range causal structure leads to broken evidence chains if we only preserve local windows around explicit errors.

To complete long-range causal chains, this stage extracts backtrackable query elements from explicit error segments and organizes them into a structured entity set E. Here, entities do not refer to arbitrary nouns in natural language, but to symbolic clues that stably appear in build logs and point to key nodes in failure chains. They include function/symbol names (e.g., `__cpuid`, `riscv_v_asm_flag`), library names (e.g., `libatomic`, `openssl-devel`), missing headers, target files and paths, build targets, macros, and key compile/linking options. The quality of E directly affects backtracking effectiveness: too few entities lead to insufficient recall, while overly noisy entities introduce irrelevant segments.

Using the entity set E as queries, we search the original log L for related segments. For each line l_j, we define a backtracking value:

$$V(l_j) = \max_{e \in E} \left(\mathrm{Sim}(l_j, e) \cdot \frac{1}{\log(1 + \lambda \Delta t(l_j, e))} \right) \tag{2}$$

where Sim denotes fuzzy matching similarity (supporting substring matches, case-insensitive matches, and small edit distances), and the distance term biases toward closer lines when similarities are comparable. Lines whose values exceed a threshold are selected and aggregated into backtracking segments, thereby filling in causal evidence scattered across build stages. For example, when an explicit error reports `undefined reference to `foo_v_probe'`, entity backtracking may locate, in earlier `configure` stages, lines like `checking for foo_v_probe... no` and the corresponding compilation failures in `config.log`, thus bridging configuration-probing failures and link-time errors.

3.1.3 Fine-Grained Semantic Compression. After coarse-grained filtering and entity backtracking, the system has converged to two types of valuable information: key error lines with their local context and related segments obtained by backtracking entities across the whole log. However, these backtracking results still exist as raw log segments that may be redundant, stylistically inconsistent, and too long, easily exceeding the context capacity of downstream models. If we simply concatenate them and feed them to an LLM, we would incur high token costs and introduce noise and duplication that harm the stability of subsequent reasoning.

Therefore, this stage performs fine-grained semantic compression. We use a lightweight, domain-tuned model to summarize and distill backtracking segments and then fuse them with explicit error information preserved in the coarse-grained stage, producing a structured and budget-controlled semantic summary of the log. Here, the summarization is task-oriented rather than generic: it focuses on information that supports RCA. On the one hand, it preserves context lines/segments helpful for localization; on the other, it attaches stage labels (e.g., `%prep`, `%build`) and distills key facts directly relevant to root causes, such as certain dependencies being disabled in `configure` due to missing packages, link-time failures like `cannot find -lxxx`, or architecture-specific symbols not being available under RISC-V.

We use Qwen3-4B-Instruct as the base model, which has good instruction-following and text-generation abilities with a moderate parameter size suitable for cost-effective deployment. We apply supervised fine-tuning (SFT) with LoRA-based parameter-efficient tuning on 2,925 manually curated log-block compression examples to guide the model toward preserving key information, compressing unhelpful details, and producing stable output formats. Experiments show that compression ratio improves from 19.66× to 30.6×, while key-error coverage increases slightly from 74.3% to 75.7%, indicating that domain-specific tuning improves compression efficiency without sacrificing essential information.

Finally, we fuse the fine-grained summaries with explicit error information from the coarse-grained stage to obtain the structured compressed result S, which is human-readable, reproducible, and well-suited for subsequent rule-based root cause reasoning.

3.2 HRL-RCA: Historical Rule Learning for Root Cause Analysis

To mitigate similarity misleading inherent in naive similarity-based retrieval and to make repair suggestions verifiable with explicit evidence, we propose Historical Rule Learning for Root Cause Analysis (HRL-RCA). The basic idea is to extract and explicitly structure the relationships among "symptoms—applicability conditions—repair actions—evidence" from historical repair records, turning scattered experience into reusable repair rules, and then build a maintainable rule-tree index along axes of problem category, build stage, and verifiable conditions. During online inference, we no longer directly retrieve by similarity; instead, we use the MGSC summary as input for conditional routing over the rule tree, retrieving rules only when their applicability conditions are satisfied, and then generating root cause explanations and fixes under explicit evidence constraints. This yields interpretable, traceable, and actionable outputs.

As shown in Figure 4, the overall workflow contains an offline construction and an online inference part. Offline, we process historical repair commits to extract and structure repair experience from commit messages, diffs/patches, and packaging-related spec changes, and store them as reusable rules. Then, we construct a rule-tree skeleton organized by error category and build stage, and attach each rule to leaf nodes whose conditions are satisfied, with conflict detection and incremental updates. Online, we take the MGSC summary S (optionally augmented with spec/patch and environment summaries) as input, first determine error type and build stage to locate the entry in the rule tree, and then traverse conditional branches to retrieve candidate leaf rules. Under evidence constraints, we finally generate a diagnostic report containing "root cause—fix—applicability conditions—evidence references".

3.2.1 Structuring Historical Repair Information. In practice, historical repair experience is mainly recorded in commits, including commit messages, code diffs/patches, and packing-related spec changes. Although they contain rich repair clues, they are inherently unstructured, stylistically inconsistent, and heterogeneous in evidence granularity: the same type of problem may be described in different ways (e.g., "fix build", "riscv workaround", "linker error"), while the actual executable repair actions are often buried in diff snippets. Directly using raw commits as retrieval targets in similarity-based methods easily reintroduces similarity misleading and cannot explicitly capture the relationship between applicability conditions and repair actions. Therefore, HRL-RCA first abstracts historical commits into structured units—repair rules—thus elevating experience from free text to computable, routable, and maintainable knowledge items.

Rule construction takes a single repair commit as the basic object, with inputs including: (1) commit message describing intent and background; (2) diffs/patches containing concrete code or script changes; (3) spec changes related to packaging (e.g., dependency declarations, macro and build-parameter modifications, patch references); and (4) associated issue/PR discussions providing information about reproduction, applicability conditions, and boundary cases. The output is a structured JSON record whose fields emphasize two aspects. First, applicability conditions are explicitly encoded (e.g., `error_type`, `build_stage`, `trigger_condition`, `risc_v_specific`,

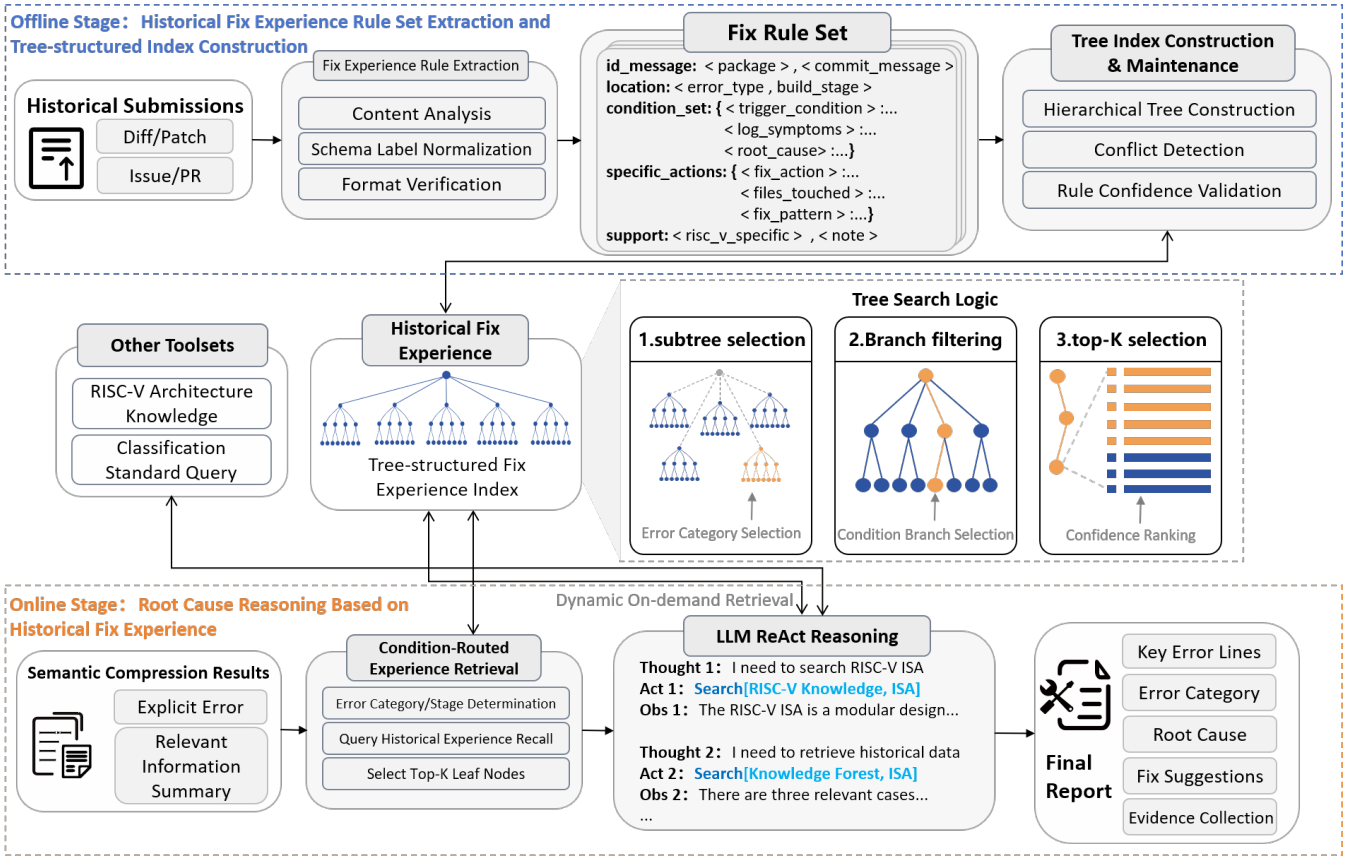

Figure 4: HRL-RCA: framework of Historical Rule Learning for Root Cause Analysis

`can_generalize`) to support later tree construction and conditional routing. Second, evidence carriers (e.g., `files_touched`, `commit_id`, `notes`) are preserved to ensure traceability and human verification.

The construction pipeline adopts a three-stage process: "traditional parsing and preprocessing—LLM-based structuring—validation and normalization". First, we use traditional techniques to pre-parse metadata and changes, including segmenting commit messages, diffs/patches, spec changes, and associated issue/PR content, and extracting salient elements such as touched files, key change hunks, dependency declarations, and macro/parameter modifications. This reduces input noise and provides structural hints for LLMs. Second, with unified prompt templates and fixed output schemas, we invoke an LLM to convert parsed results into structured repair rules, covering fields such as `error_type`, `build_stage`, `trigger_condition`, `root_cause`, and `fix_action`, and mapping key fields such as triggers and fix patterns to predefined label sets whenever possible. Finally, an independent validation module automatically checks LLM outputs: it verifies schema conformance (field completeness, type consistency, non-empty required fields) and performs normalization consistency checks for `fix_action` and `trigger_condition`. When validation fails, the system triggers a rewrite strategy, asking the model to regenerate specific fields using feedback from validation, ensuring that only rules with stable formats, routable conditions, and traceable evidence enter the knowledge base.

3.2.2 Rule-Tree Index Construction and Maintenance. After obtaining structured rules, we need to organize scattered repair experience into a routable and maintainable knowledge structure. We use a tree index for knowledge organization for three reasons. First, the manual diagnostic process for build failures essentially follows a decision path of first determining error category and build stage, and then narrowing down by applicability conditions. A tree index explicitly encodes this process as conditional branches, making rule-matching paths naturally interpretable and reviewable. Second, the effectiveness of repair experience strongly depends on preconditions (e.g., versions, architecture, dependency states, build systems, stages). Tree routing requires each step to follow verifiable conditions, filtering out inapplicable rules at the retrieval entrance and thus mitigating risks of error-prone retrieval due to pure similarity. Third, if we simply feed all rules and their conditions to an LLM for conditional retrieval, token costs would be unmanageable, and the model would easily suffer from attention dispersion and missed conditions amid redundancy. The tree index pre-materializes the routing structure, performing structural pruning ahead of LLM inference and injecting only a small number of rules that satisfy the conditions, thus enabling small-context, strongly constrained, and reviewable reasoning.

As illustrated in Figure 5, we start from a root node and split the first layer into five domains in accordance with our error taxonomy (architecture compatibility, version compatibility, dependency configuration, build configuration, and other errors), so that online inference can first perform coarse routing at the problem-domain level. The second layer is organized by build stage (e.g., `%prep`, `%build`, `%install`, `%check`), constraining historical experience to consistent stage contexts to avoid misinterpretation of the same error text under different stages. Deeper nodes branch according to verifiable routing conditions derived from `trigger_condition` and `log_symptoms` in rules, such as "whether dependency xxx is detected missing", "whether the failure only occurs on RISC-V", etc. Leaf nodes store sets of concrete repair rules with their `fix_action`, applicability conditions, and annotations. Once a path is hit during online inference, the system obtains a joint package of "conditions—actions—evidence".

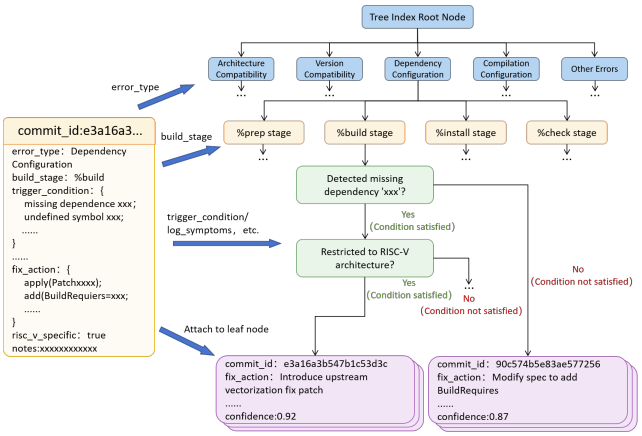

Figure 5: Illustration of the rule-tree index structure

To support continuous evolution, we design incremental updates and conflict-resolution mechanisms. Given a new rule, we first normalize it using the same offline procedure and then determine its routing position. If the condition set C_{new} of the new rule and C_{old} of an existing rule have inclusion relationships, we handle them as follows: when $C_{new} \supset C_{old}$, the new rule has stricter conditions, so we add a deeper branch in the tree and give the fine-grained rule higher priority; when $C_{new} \subset C_{old}$, the new rule is more general, and we adjust the tree to generalize conditions and reduce duplicate branches. This avoids naive stacking of rules that would lead to path explosion.

We further introduce rule confidence for quality control:

$$\mathrm{Conf}(r) = \alpha \cdot S_{evi}(r) + (1-\alpha) \cdot S_{time}(r) \tag{3}$$

where $S_{evi}(r) = n_r/(n_r + k)$ is evidence support (with n_r being the number of supporting records and k a smoothing constant, default k=3), and $S_{time}(r) = 1/(1 + (t_{now} - t_r)/T)$ measures recency (with T=90 days, meaning the recency drops to 0.5 after 90 days). Rules whose confidence falls below a threshold are used only as low-priority candidates and placed in a queue for manual review; only those above the threshold become high-priority leaf evidence for routing. This mechanism jointly constrains knowledge quality and freshness during incremental evolution.

3.2.3 Online Root Cause Reasoning. In the online phase, given a build-failure case, the goal is to perform conditional routing and historical-experience retrieval along the rule tree, and then, under explicit evidence constraints, generate interpretable, actionable, and verifiable root cause analyses and fixes. We adopt a ReAct-style evidence-driven reasoning framework, using the MGSC summary S as the entry, and encapsulating external knowledge queries and rule-tree retrieval as tools that the model can invoke. The model then iterates between "acting" and "observing" to gradually collect evidence and converge on applicable rules.

Error type and stage determination. The first step is to determine the error category and build stage of the current failure. The MGSC summary S already includes explicit error lines, stage-related clues, key entities, and several evidence snippets. If S explicitly contains stage information, we use it directly; otherwise, we infer the stage based on typical patterns around error anchors. For error categories, we map error anchors and entities in S to one of our five categories via a predefined standard and conditions. When S reveals architecture-specific entities or patterns, we optionally call RISC-V knowledge tools for additional checks to avoid misclassification based solely on surface error messages.

Retrieving relevant historical repair experience. After determining error type and stage, we query the rule tree. We first locate the problem-domain node according to `error_type` and then enter the corresponding stage subtree according to `build_stage`, restricting candidates to a semantically coherent region. Inside the stage subtree, we use triggers and key entities extracted from S to match against branch conditions. We traverse top-down: when evidence clearly satisfies a branch, we follow it; when evidence is insufficient, we retain the branch as a candidate to avoid premature pruning; when evidence clearly contradicts a branch, we skip it. If multiple leaf nodes remain, we rank their rules by matching degree (coverage of triggers and entities) and rule confidence, and keep the Top-K rules as candidate evidence. If no leaves are hit, we back off to higher-level nodes and fall back to more general rules.

ReAct-style reasoning. After retrieving candidate rules, we proceed to final answer generation. We pack three kinds of information into the LLM context: (1) the MGSC summary S, providing error anchors, stage clues, and entities; (2) retrieved historical rules (Top-K candidates); and (3) routing path information (problem domain, stage, and branch sequence). Following the ReAct paradigm, the model first forms preliminary hypotheses based on S and candidate rules. If it detects missing critical information or inconsistencies among candidates, it issues "actions" to call tools (e.g., RISC-V knowledge queries or validation of hypothetical fixes), uses their outputs as "observations", and then revises its hypotheses. Finally, the model is required to output a structured diagnostic report containing: (1) key error and stage localization; (2) error-type classification and rationale; (3) verifiable root cause explanation; (4) concrete, executable repair suggestions; and (5) supporting evidence and references to historical repairs. When multiple candidate rules remain plausible, the system outputs a primary recommendation and alternatives, each with its `trigger_condition` specifying applicability boundaries, thus avoiding unverifiable compromises when evidence conflicts.

4 Evaluation

4.1 Experimental Setup

4.1.1 Datasets. Our experimental data is drawn from RISC -V package build-failure logs in a real OBS production environment. For log compression, we use build logs of 133 different packages, totaling 149,219 log lines. Domain experts label 4,582 key repair-related lines as ground truth. For root cause analysis, we use 174 representative build-failure cases covering five error types (architecture compatibility, build configuration, dependency configuration, version compatibility, and other errors).

As shown in Figure 6, version-compatibility errors account for the largest proportion (29.9%), while build-configuration and dependency-configuration errors each account for 23.6%, other errors 14.4%, and architecture-compatibility errors 8.6%. Regarding build-stage distribution, 50.0% of failures occur in `%build`, 24.1% in `%prep`, 20.7% in `%check`, and 5.2% in `%install`.

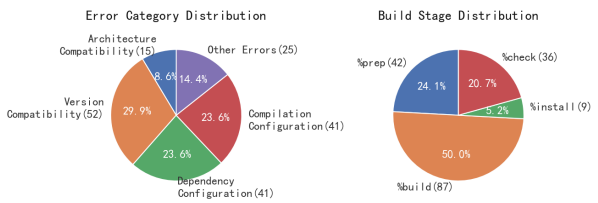

Figure 6: Distribution of error categories and build stages in the RISC -V build-failure dataset

4.1.2 Baselines. For log compression, we compare MGSC with three baselines: (1) LogSage [20], an LLM-based analysis framework for long logs; (2) TemplateRAG [16], a template-based parsing and retrieval-augmented method; and (3) LogPrompt [9], which enhances LLM-based log analysis via prompt engineering.

For root cause analysis, we consider four classes of baselines: (1) a naive RAG agent based on semantic-similarity retrieval and generation; (2) LogSage [20], which uses improved multi-path RAG retrieval; (3) MCTS [16], a search-based reasoning framework; and (4) flagship reasoning models (Seed1.8, Kimi K2, DeepSeekV3.2), which primarily rely on model capacity improvements.

4.1.3 Metrics. For log compression, we use line-level Precision/Recall/F1 to evaluate the preservation of key evidence. For root cause analysis, we adopt a composite score:

$$\text{score}(i) = \frac{s_i + c_i + r_i + f_i}{4} \tag{4}$$

where s_i is the score for extracting key error statements, c_i for error classification, r_i for root cause explanation, and f_i for repair suggestion.

Table 1: Accuracy comparison of log compression methods

Method	Prec	Rec	F1
LogPrompt	0.260	0.300	0.334
TemplateRAG	0.047	**0.769**	0.080
LogSage	**0.308**	0.574	0.387
MGSC (ours)	0.302	0.668	**0.389**

4.2 Results on Log Compression

As shown in Table 1, MGSC achieves a Recall of 0.668, significantly higher than LogSage (0.574), indicating that the combination of error-anchor-based coarse filtering and entity backtracking can more effectively recover key clues along long-range causal chains. At the same time, MGSC attains much higher Precision than TemplateRAG (0.302 vs. 0.047), showing that MGSC can suppress large amounts of irrelevant noise while improving coverage. In terms of efficiency, MGSC's average output tokens for three groups of logs are 3442/4234/5502, consistently at the scale of a few thousand tokens and one order of magnitude smaller than TemplateRAG (38746/49883/55406), leaving ample context budget for subsequent reasoning. Regarding runtime, MGSC achieves about 2.3–3.6× speedup over LogPrompt and TemplateRAG on ultra-long logs.

4.3 Ablation Study on MGSC

We perform an ablation study to assess the contribution of each MGSC component.

Table 2: Ablation results of MGSC

Configuration	Prec	Rec	F1
MGSC (full)	0.302	0.668	0.389
w/o Anchor Filter	0.274	0.352	0.328
w/o Entity Backtracking	0.292	0.584	0.372
w/o Semantic Compressor	0.246	0.668	0.349

As shown in Table 2, removing error-anchor-based coarse filtering (w/o Anchor Filter) reduces Recall from 0.668 to 0.352 (a 47.3% drop) and decreases F1 by 15.7%, demonstrating that explicit anchor localization is crucial for quickly focusing on abnormal regions. Removing entity backtracking (w/o Entity Backtracking) reduces Recall and F1 by 12.6% and 4.4%, respectively, indicating that this module effectively recovers long-distance causal clues. Without the semantic compressor (w/o Semantic Compressor), Recall remains the same but Precision drops by 18.5%, showing that the lightweight compressor mainly improves output purity and stability of the structured summaries.

4.4 Results on Root Cause Analysis

As shown in Table 3, HRL-RCA consistently achieves the best comprehensive scores across both backbone groups. With DeepSeekV3 as the backbone, HRL-RCA reaches an overall score of 0.790, outperforming the strongest baseline MCTS (0.735) by 5.5% and the naive RAG agent (0.650) by 21.5%.

Figure 7 shows that HRL-RCA's main advantages lie in repair recommendation and root cause explanation. With DeepSeekV3,

Table 3: Overall performance comparison on root cause analysis

Group	Method	s_i	c_i	r_i	f_i	Score
DeepSeekV3	RAG	0.769	0.657	0.655	0.520	0.650
	LogSage	0.870	0.754	0.692	0.524	0.710
	MCTS	0.881	0.777	0.714	0.567	0.735
	HRL-RCA	**0.892**	**0.798**	**0.766**	**0.706**	**0.790**
GPT-OSS	RAG	0.743	0.625	0.693	0.613	0.669
	LogSage	0.791	0.699	0.721	0.606	0.704
	MCTS	**0.841**	**0.747**	0.754	0.590	0.733
	HRL-RCA	0.821	0.745	**0.760**	**0.742**	**0.767**
Reasoning Models	Seed 1.8	0.814	0.665	0.755	0.635	0.717
	Kimi K2	0.833	0.713	0.744	0.592	0.721
	DSV3.2	0.728	0.605	0.691	0.545	0.642

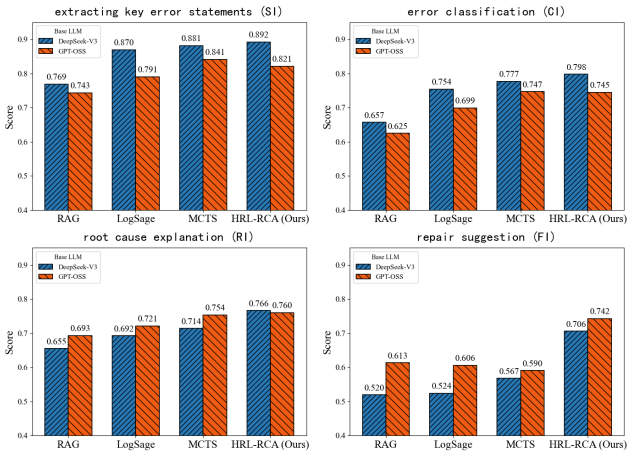

Figure 7: Per-metric score comparison across methods

HRL-RCA attains a repair-suggestion score of 0.706, surpassing MCTS (0.567) by 0.139. This indicates that conditional routing and evidence binding based on historical repair rules make it easier to recall executable fixes under satisfied preconditions. Scores for key-error extraction are relatively high across methods (0.769–0.892), suggesting that unified log compression already provides strong error anchors; HRL-RCA still yields the highest score (0.892), reflecting the stabilizing effect of structured reasoning paths.

Compared with standalone flagship reasoning models, Seed1.8, Kimi K2, and DeepSeekV3.2 all perform worse than HRL-RCA. This suggests that for build failures, where applicability conditions and verifiable evidence are critical, merely increasing model capacity is insufficient to ensure high-quality repair suggestions; structured knowledge organization and evidence constraints are necessary.

4.5 Efficiency Analysis

As shown in Figure 8, with DeepSeekV3 as the backbone, HRL-RCA has an average runtime of 101.80s, reducing processing time by about 61.0% compared with MCTS (261.23s) and being comparable to LogSage (86.93s). The naive RAG baseline is the fastest (32.64s) but yields significantly lower accuracy. MCTS is the slowest due to multi-round search-based reasoning, whereas HRL-RCA

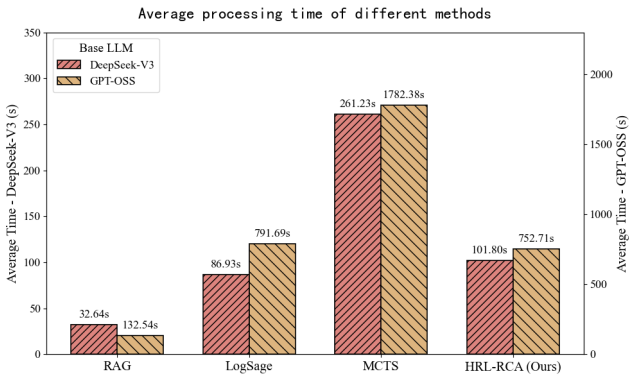

Figure 8: Average processing time of different methods

constrains candidate space via rule routing, avoiding the overhead of global search and achieving the best overall accuracy while remaining practically efficient.

5 Conclusion and Future Work

This paper targets build failures in RISC -V package migration and proposes a two-stage root cause analysis method that combines log compression with rule learning. In the first stage, MGSC uses explicit error anchor localization, entity-based long-distance context backtracking, and lightweight semantic compression to preserve high-fidelity evidence chains under strict token budgets, significantly reducing long-log input size. In the second stage, HRL-RCA structures historical repair experience into verifiable rules organized as a rule tree and uses conditional routing during online inference to retrieve rules whose preconditions hold, generating traceable root cause analyses and repair suggestions under explicit evidence constraints.

Experimental results show that MGSC achieves higher coverage of key evidence than existing compression methods while keeping output token size stable and controllable. HRL-RCA attains a comprehensive diagnostic score of 79.0%, significantly outperforming naive RAG, multi-path retrieval-augmented methods, and search-based reasoning baselines, while reducing runtime by 61% compared with MCTS, thus offering clear advantages in both effectiveness and efficiency. Future work will further improve incremental updates and deduplication for the rule knowledge base, explore adapting the method to more architectures and build systems, and develop IDE plugins and other developer tools to facilitate adoption in real-world engineering workflows.

Acknowledgments

This work was supported in part by projects of the Institute of Software, Chinese Academy of Sciences, among others.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[2] Song Chen and Hai Liao. 2022. Bert-log: Anomaly detection for system logs based on pre-trained language model. *Applied Artificial Intelligence* 36, 1 (2022),

2145642.

[3] Enfang Cui, Tianzheng Li, and Qian Wei. 2023. Risc-v instruction set architecture extensions: A survey. *IEEE Access* 11 (2023), 24696–24711.

[4] Hetong Dai, Heng Li, Che-Shao Chen, Weiyi Shang, and Tse-Hsun Chen. 2020. Logram: Efficient log parsing using *n* n-gram dictionaries. *IEEE transactions on software engineering* 48, 3 (2020), 879–892.

[5] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. 2017. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*. 1285–1298.

[6] Haixuan Guo, Shuhan Yuan, and Xintao Wu. 2021. Logbert: Log anomaly detection via bert. In *2021 international joint conference on neural networks (IJCNN)*. IEEE, 1–8.

[7] Pinjia He, Jieming Zhu, Zibin Zheng, and Michael R Lyu. 2017. Drain: An online log parsing approach with fixed depth tree. In *2017 IEEE international conference on web services (ICWS)*. IEEE, 33–40.

[8] Jonathan Lee, Mason Li, and Kuo-Hsun Hsu. 2024. Applying transformer models for automatic build errors classification of java-based open source projects. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*. 282–283.

[9] Yilun Liu, Shimin Tao, Weibin Meng, Feiyu Yao, Xiaofeng Zhao, and Hao Yang. 2024. Logprompt: Prompt engineering towards zero-shot and interpretable log analysis. In *Proceedings of the 2024 IEEE/ACM 46th international conference on software engineering: Companion proceedings*. 364–365.

[10] Yiling Lou, Zhenpeng Chen, Yanbin Cao, Dan Hao, and Lu Zhang. 2020. Understanding build issue resolution in practice: symptoms and fix patterns. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 617–628.

[11] Zeyang Ma, An Ran Chen, Dong Jae Kim, Tse-Hsun Chen, and Shaowei Wang. 2024. Llmparser: An exploratory study on using large language models for log parsing. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.

[12] Benjamin W Mezger, Douglas A Santos, Luigi Dilillo, Cesar A Zeferino, and Douglas R Melo. 2022. A survey of the RISC-V architecture software support. *IEEE Access* 10 (2022), 51394–51411.

[13] Paolo Notaro, Soroush Haeri, Jorge Cardoso, and Michael Gerndt. 2023. Logrule: Efficient structured log mining for root cause analysis. *IEEE Transactions on Network and Service Management* 20, 4 (2023), 4231–4243.

[14] Thomas Rausch, Waldemar Hummer, Philipp Leitner, and Stefan Schulte. 2017. An empirical analysis of build failures in the continuous integration workflows of java-based open-source software. In *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. IEEE, 345–355.

[15] Devjeet Roy, Xuchao Zhang, Rashi Bhave, Chetan Bansal, Pedro Las-Casas, Rodrigo Fonseca, and Saravan Rajmohan. 2024. Exploring llm-based agents for root cause analysis. In *Companion proceedings of the 32nd ACM international conference on the foundations of software engineering*. 208–219.

[16] Weipeng Shuai, Jie Liu, Zhirou Ma, Liangyi Kang, Zehua Wang, Shuai Wang, Dan Ye, Hui Li, Wei Wang, and Jiaxin Zhu. 2025. Root Cause Analysis of RISC-V Build Failures via LLM and MCTS Reasoning. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2772–2782.

[17] Risto Vaarandi. 2008. Mining event logs with slct and loghound. In *NOMS 2008-2008 IEEE Network Operations and Management Symposium*. IEEE, 1071–1074.

[18] Thorsten Wittkopp, Philipp Wiesner, and Odej Kao. 2024. Logrca: Log-based root cause analysis for distributed services. In *European Conference on Parallel Processing*. Springer, 362–376.

[19] Junjielong Xu, Ruichun Yang, Yintong Huo, Chengyu Zhang, and Pinjia He. 2024. Divlog: Log parsing with prompt enhanced in-context learning. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–12.

[20] Weiyuan Xu, Juntao Luo, Tao Huang, Kaixin Sui, Jie Geng, Qijun Ma, Isami Akasaka, Xiaoxue Shi, Jing Tang, and Peng Cai. 2025. LogSage: An LLM-based framework for CI/CD failure detection and remediation with industrial validation. *arXiv preprint arXiv:2506.03691* (2025).

[21] Ding Yuan, Haohui Mai, Weiwei Xiong, Lin Tan, Yuanyuan Zhou, and Shankar Pasupathy. 2010. Sherlog: error diagnosis by connecting clues from run-time logs. In *Proceedings of the fifteenth International Conference on Architectural support for programming languages and operating systems*. 143–154.